

Microsoft Fabric Warehouse

Security & Permissions

DBA Technical Reference | Volume 3 of 9

April 2025 | Version 1.0

Table of Contents

1.	Security Architecture Overview	3
2.	Authentication Methods	3
3.	Workspace-Level Roles	4
4.	Warehouse-Level Permissions	4
5.	Object-Level Permissions (GRANT/REVOKE/DENY)	5
6.	Row-Level Security (RLS)	6
7.	Column-Level Security	6
8.	Service Principals & Managed Identities	7
9.	Data Encryption	7
10.	Audit & Compliance	8
11.	Security Best Practices	8

1. Security Architecture Overview

Fabric Warehouse implements a multi-layered security model. Permissions cascade from the broadest scope (tenant) down to the most granular (individual columns). Understanding this hierarchy is critical to correctly restricting access.

Security Layer	Scope	Managed By
Tenant	Entire Fabric environment	Fabric / Microsoft 365 Admin
Capacity	All workspaces in a capacity	Capacity Administrator
Workspace	All items in a workspace	Workspace Admin
Warehouse	Specific warehouse database	Workspace Admin / Member
Schema	All objects in a schema	DBA / data owner
Object	Individual table, view, procedure	DBA / data owner
Row	Specific rows within a table	RLS policy (T-SQL)
Column	Specific columns within a table	Column permission (T-SQL)

WARNING: Granting broad workspace-level roles (Admin, Member) gives users extensive permissions across ALL items in the workspace. Always prefer object-level GRANT where possible.

2. Authentication Methods

Fabric Warehouse uses Azure Active Directory (Entra ID) as the identity provider. SQL authentication (username/password) is not supported.

Authentication Type	Description	Use Case
Azure AD User Account	Interactive login with MFA	Human users, developers, DBAs
Service Principal	App registration with client ID + secret or certificate	Automated pipelines, CI/CD
Managed Identity	Azure resource identity — no credentials to manage	Azure Data Factory, Azure Functions
Azure AD Group	Group membership grants access	Team-based access control

>> Connect Using Service Principal (Python)

```
import pyodbc

conn_str = (
'DRIVER={ODBC Driver 18 for SQL Server};'
'SERVER=.datawarehouse.fabric.microsoft.com;'
'DATABASE=;'
'Authentication=ActiveDirectoryServicePrincipal;'
'UID=;'
'PWD=;'
)

conn = pyodbc.connect(conn_str)
```

3. Workspace-Level Roles

Role	Read Data	Write Data	Manage Items	Admin Actions
Viewer	Yes (via SQL endpoint)	No	No	No
Contributor	Yes	Yes	No	No
Member	Yes	Yes	Yes	No
Admin	Yes	Yes	Yes	Yes (incl. KILL)

INFO: Workspace roles apply to ALL warehouses, lakehouses, and other items within the workspace. Use them sparingly and prefer per-warehouse object permissions for fine-grained control.

4. Warehouse-Level Permissions

In addition to workspace roles, you can grant specific permissions directly on a Warehouse item. These permissions are managed through the Fabric portal or T-SQL.

Permission	Allows	T-SQL Grant
ReadData	SELECT on all tables and views	GRANT SELECT ON SCHEMA::dbo TO [user]
ReadAll	Read all objects including metadata	Via Fabric portal item permissions
Write	INSERT, UPDATE, DELETE on all tables	GRANT INSERT, UPDATE, DELETE ON SCHEMA::dbo TO [user]
Execute	Execute stored procedures	GRANT EXECUTE ON SCHEMA::dbo TO [user]
ViewDefinition	View object definitions (DDL)	GRANT VIEW DEFINITION TO [user]

5. Object-Level Permissions (GRANT / REVOKE / DENY)

Object-level permissions provide the most granular access control. These are applied using standard T-SQL statements.

>> Grant SELECT on Specific Table

```
-- Allow a user to read a specific table only
GRANT SELECT ON dbo.FactSales TO [analyst@contoso.com];
```

>> Grant Multiple Permissions on Schema

```
-- Allow a service principal to read and write all objects in dbo schema
GRANT SELECT, INSERT, UPDATE, DELETE
ON SCHEMA::dbo
TO [my-service-principal];
```

>> Revoke a Permission

```
-- Remove SELECT permission from a user
REVOKE SELECT ON dbo.FactSales FROM [analyst@contoso.com];
```

>> Deny Access (Overrides GRANT)

```
-- Explicitly deny access to sensitive table (overrides any GRANT)
DENY SELECT ON dbo.SensitiveData TO [analyst@contoso.com];
```

>> Grant EXECUTE on Stored Procedure

```
-- Allow user to run a specific stored procedure
GRANT EXECUTE ON dbo.usp_GetSalesReport TO [report_user@contoso.com];
```

>> View Current Object Permissions

```
SELECT
pr.name AS principal_name,
pr.type_desc AS principal_type,
pe.state_desc AS permission_state,
pe.permission_name,
OBJECT_NAME(pe.major_id) AS object_name
FROM sys.database_permissions pe
JOIN sys.database_principals pr ON pe.grantee_principal_id = pr.principal_id
WHERE pe.class = 1 -- object-level permissions
ORDER BY pr.name, pe.permission_name;
```

6. Row-Level Security (RLS)

Row-Level Security restricts which rows a user can see when querying a table. It is implemented using security predicates — inline functions that filter rows based on the calling user's identity.

>> Create an RLS Filter Function

```
-- Create security predicate function
CREATE FUNCTION dbo.fn_SecurityPredicate (@Region NVARCHAR(50))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS result
WHERE @Region = USER_NAME()
OR IS_MEMBER('db_owner') = 1;
GO

-- Apply the security policy to the table
CREATE SECURITY POLICY SalesRegionPolicy
ADD FILTER PREDICATE dbo.fn_SecurityPredicate(Region)
ON dbo.FactSales
WITH (STATE = ON);
```

INFO: RLS policies are transparent to the query — users see only their permitted rows without any change to their queries. Admins with db_owner rights bypass RLS by default.

7. Column-Level Security

Column-level security prevents specific users from reading sensitive columns such as salaries, SSNs, or PII data, without hiding the rest of the table.

>> Deny Access to Specific Columns

```
-- Prevent analyst from reading salary and SSN columns
DENY SELECT ON dbo.Employees (Salary, SSN, BankAccount)
TO [analyst@contoso.com];
```

>> Grant Access to Only Specific Columns

```
-- Grant access at table level, then deny sensitive columns
GRANT SELECT ON dbo.Employees TO [analyst@contoso.com];
DENY SELECT ON dbo.Employees (Salary, SSN) TO [analyst@contoso.com];
```

8. Service Principals & Managed Identities

Identity Type	Best For	Secret Management
Service Principal + Secret	Pipelines, apps needing specific auth	Rotate secrets regularly; store in Key Vault
Service Principal + Certificate	Higher security than secret-based	Certificate management required
Managed Identity	Azure-hosted services (ADF, Functions)	No secrets — identity managed by Azure
Azure AD Group	Team access management	Add/remove members without permission changes

```
>> Create a Database User for a Service Principal

-- Create user mapped to external provider (Entra ID / service principal)
CREATE USER [my-pipeline-spn] FROM EXTERNAL PROVIDER;

-- Grant required permissions
GRANT SELECT, INSERT ON SCHEMA::dbo TO [my-pipeline-spn];
```

9. Data Encryption

Encryption Type	Details
Encryption at Rest	All OneLake data (Parquet/Delta files) encrypted using AES-256 by default
Encryption in Transit	All connections use TLS 1.2+ — enforced, cannot be disabled
Customer-Managed Keys (CMK)	Optional: bring your own encryption key stored in Azure Key Vault
CMK During Failover	CMK supports read-only operations during regional failover as long as Key Vault is healthy

10. Audit & Compliance

Fabric integrates with Microsoft Purview for governance and audit capabilities. Query-level audit trails are available through Query Insights DMVs.

```
>> Audit: Who Ran What and When

SELECT
    login_name,
    start_time,
    end_time,
    status,
    row_count,
    command
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY, -30, GETUTCDATE())
ORDER BY start_time DESC;
```

11. Security Best Practices

- Never assign Workspace Admin or Member roles to regular data consumers — use Viewer or object-level GRANT
- Use Azure AD Groups for access management — avoids modifying T-SQL permissions for each person
- Implement Row-Level Security for multi-tenant or region-segmented data warehouses
- Store service principal secrets in Azure Key Vault — never hardcode credentials
- Rotate service principal secrets and certificates on a scheduled basis (every 90 days recommended)
- Enable Microsoft Purview integration for data classification and governance at scale
- Audit query history monthly using queryinsights.exec_requests_history to detect anomalous access patterns

- Apply DENY on sensitive columns (PII, financial data) as a defense-in-depth measure
- Use Managed Identity over Service Principal + Secret wherever the workload runs on Azure
- Document all permission grants and review quarterly for least-privilege compliance